Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №8
Варіант 9
На тему
“Алгебраїчні типи даних «по-Python'івськи»”

Виконав:
Студент групи ФЕП-22с
Обиход І. В.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- змоделювати алгебраїчний тип даних через Union/TypeAlias і frozen dataclass;
- використати Literal і Protocol;
- застосувати структурний pattern matching match/case для розбору структур;
- забезпечити проходження mypy --strict.

Теоретичні відомості:

- Алгебраїчний тип даних (sum type) - тип, що є одним із кількох варіантів; у Python моделюється класами та Union.
- Literal звужує значення до конкретних літералів, Protocol описує структурний інтерфейс (duck typing з перевіркою mypy).
- match/case розбирає структуру за класовими шаблонами, наприклад case JArray(items=items).
- frozen=True робить вузли незмінними, а assert_never дозволяє mypy перевірити вичерпність match.

Завдання (варіант 9 - JSON-подібне дерево):

JSON-подібне дерево JNull, JBool, JNumber, JString, JArray, JObject. Реалізувати рендер і підрахунок кількості вузлів. ADT із рекурсивними TypeAlias; pattern matching для всіх JSON-типів.

Хід роботи

У model.py шість вузлів описано як @dataclass(frozen=True, slots=True) і об'єднано в рекурсивний тип type JSON = JNull | JBool | JNumber | JString | JArray | JObject. JArray містить tuple вузлів, JObject - tuple пар (ключ, вузол). Тип вузла описано через Kind = Literal[...], а Protocol SupportsToJSON задає структурний контракт для будь-яких об'єктів із методом to_json().

У evaluator.py операції реалізовано чистими функціями з match/case: kind, count_nodes, depth, stats (кількість вузлів кожного типу), to_python, from_python, get (доступ за шляхом) і with_field (pure update, що повертає новий JObject). Вичерпність match контролює assert_never.

У pretty.py реалізовано render (компактний JSON), pretty (з відступами) і tree (текстова структура з типами вузлів).

from_python розпізнає None, bool, числа, рядки, списки, словники та об'єкти з протоколом SupportsToJSON, а для непідтримуваних типів і нерядкових ключів дає TypeError.

Тести перевіряють кожен варіант ADT, збіг render і pretty з json.dumps на вкладеній структурі, round-trip перетворення, підрахунок вузлів, глибину, статистику, незмінність вузлів, протокол і помилкові випадки.

Файл model.py

model.py рядки 1-61

```python
"""ADT JSON-подібного дерева (ЛР8, варіант 9).

JSON = JNull | JBool | JNumber | JString | JArray | JObject - сума варіантів,
кожен вузол - незмінний frozen dataclass. JArray та JObject рекурсивні.
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Literal, Protocol, runtime_checkable

Kind = Literal["null", "bool", "number", "string", "array", "object"]


@dataclass(frozen=True, slots=True)
class JNull:
    """JSON null."""


@dataclass(frozen=True, slots=True)
class JBool:
    """JSON true/false."""

    value: bool


@dataclass(frozen=True, slots=True)
class JNumber:
    """JSON число."""

    value: int | float


@dataclass(frozen=True, slots=True)
class JString:
    """JSON рядок."""

    value: str


@dataclass(frozen=True, slots=True)
class JArray:
    """JSON масив: незмінний tuple вузлів."""

    items: tuple[JSON, ...] = ()


@dataclass(frozen=True, slots=True)
class JObject:
    """JSON об'єкт: незмінний tuple пар (ключ, вузол) зі збереженням порядку."""

    fields: tuple[tuple[str, JSON], ...] = ()


type JSON = JNull | JBool | JNumber | JString | JArray | JObject


@runtime_checkable
class SupportsToJSON(Protocol):
    """Структурний контракт: будь-який об'єкт з методом to_json()."""
```

model.py рядки 62-64

```
62      def to_json(self) -> JSON:
63          """Повертає JSON-дерево, що представляє об'єкт."""
64          ...
```

Файл evaluator.py

evaluator.py рядки 1-59

```python
"""Чисті операції над JSON-деревом через match/case (ЛР8, варіант 9)."""

from collections import Counter
from functools import reduce
from typing import assert_never

from model import (
    JSON,
    JArray,
    JBool,
    JNull,
    JNumber,
    JObject,
    JString,
    Kind,
    SupportsToJSON,
)


def kind(node: JSON) -> Kind:
    """Тип вузла як Literal."""
    match node:
        case JNull():
            return "null"
        case JBool():
            return "bool"
        case JNumber():
            return "number"
        case JString():
            return "string"
        case JArray():
            return "array"
        case JObject():
            return "object"
        case _:
            assert_never(node)


def children(node: JSON) -> tuple[JSON, ...]:
    """Безпосередні дочірні вузли."""
    match node:
        case JArray(items=items):
            return items
        case JObject(fields=fields):
            return tuple(value for _, value in fields)
        case _:
            return ()


def count_nodes(node: JSON) -> int:
    """Кількість вузлів дерева (рекурсивно)."""
    return 1 + sum(count_nodes(child) for child in children(node))


def depth(node: JSON) -> int:
    """Глибина дерева: лист має глибину 1."""
    return 1 + max((depth(child) for child in children(node)), default=0)
```

evaluator.py рядки 60-118

```python
def stats(node: JSON) -> dict[Kind, int]:
    """Кількість вузлів кожного типу."""
    counts = reduce(
        lambda acc, child: acc + Counter(stats(child)),
        children(node),
        Counter({kind(node): 1}),
    )
    return dict(counts)


def to_python(node: JSON) -> object:
    """Перетворює дерево на звичайні Python-значення."""
    match node:
        case JNull():
            return None
        case JBool(value=b):
            return b
        case JNumber(value=n):
            return n
        case JString(value=s):
            return s
        case JArray(items=items):
            return [to_python(item) for item in items]
        case JObject(fields=fields):
            return {key: to_python(value) for key, value in fields}
        case _:
            assert_never(node)


def from_python(value: object) -> JSON:
    """Будує дерево з Python-значення або об'єкта з протоколом SupportsToJSON."""
    match value:
        case None:
            return JNull()
        case bool(b):
            return JBool(b)
        case int(n) | float(n):
            return JNumber(n)
        case str(s):
            return JString(s)
        case list() | tuple():
            return JArray(tuple(from_python(item) for item in value))
        case dict():
            return JObject(tuple(_field(key, item) for key, item in value.items()))
        case SupportsToJSON():
            return value.to_json()
        case _:
            raise TypeError(f"Непідтримуваний тип: {type(value).__name__}")


def _field(key: object, item: object) -> tuple[str, JSON]:
    """Пара ключ-значення для JObject; ключ має бути рядком."""
    match key:
        case str(name):
            return name, from_python(item)
        case _:
            raise TypeError(f"Ключ об'єкта має бути рядком, отримано {key!r}")
```

evaluator.py рядки 119-139

```python
def get(node: JSON, *path: str | int) -> JSON | None:
    """Значення за шляхом з ключів та індексів або None."""
    if not path:
        return node
    head, *rest = path
    match node, head:
        case JObject(fields=fields), str(key):
            found = next((v for k, v in fields if k == key), None)
            return None if found is None else get(found, *rest)
        case JArray(items=items), int(index) if -len(items) <= index < len(items):
            return get(items[index], *rest)
        case _:
            return None


def with_field(obj: JObject, key: str, value: JSON) -> JObject:
    """Повертає НОВИЙ об'єкт з доданим або заміненим полем."""
    kept = tuple((k, v) for k, v in obj.fields if k != key)
    if len(kept) == len(obj.fields):
        return JObject((*obj.fields, (key, value)))
    return JObject(tuple((k, value if k == key else v) for k, v in obj.fields))
```

Файл pretty.py

pretty.py рядки 1-62

```python
"""Текстове представлення JSON-дерева (ЛР8, варіант 9)."""

import json
from typing import assert_never

from model import JSON, JArray, JBool, JNull, JNumber, JObject, JString


def render(node: JSON) -> str:
    """Компактний JSON-текст."""
    match node:
        case JNull():
            return "null"
        case JBool(value=b):
            return "true" if b else "false"
        case JNumber(value=n):
            return json.dumps(n)
        case JString(value=s):
            return json.dumps(s, ensure_ascii=False)
        case JArray(items=items):
            return "[" + ",".join(render(item) for item in items) + "]"
        case JObject(fields=fields):
            pairs = (
                f"{json.dumps(k, ensure_ascii=False)}:{render(v)}" for k, v in fields
            )
            return "{" + ",".join(pairs) + "}"
        case _:
            assert_never(node)


def pretty(node: JSON, indent: int = 2, level: int = 0) -> str:
    """Багаторядковий JSON-текст з відступами."""
    pad, inner = " " * (indent * level), " " * (indent * (level + 1))
    match node:
        case JArray(items=items) if items:
            lines = (inner + pretty(item, indent, level + 1) for item in items)
            return "[\n" + ",\n".join(lines) + "\n" + pad + "]"
        case JObject(fields=fields) if fields:
            lines = (
                f"{inner}{json.dumps(k, ensure_ascii=False)}: "
                f"{pretty(v, indent, level + 1)}"
                for k, v in fields
            )
            return "{\n" + ",\n".join(lines) + "\n" + pad + "}"
        case _:
            return render(node)


def tree(node: JSON, label: str = "root", level: int = 0) -> str:
    """Структура дерева з типами вузлів, по одному вузлу в рядку."""
    prefix = "  " * level
    match node:
        case JArray(items=items):
            head = f"{prefix}{label}: array[{len(items)}]"
            rows = (tree(item, f"[{i}]", level + 1) for i, item in enumerate(items))
            return "\n".join((head, *rows))
        case JObject(fields=fields):
            head = f"{prefix}{label}: object{{{len(fields)}}}"
            rows = (tree(value, key, level + 1) for key, value in fields)
            return "\n".join((head, *rows))
        case _:
            return f"{prefix}{label}: {render(node)}"
```

Файл main.py

main.py рядки 1-54

```python
"""Точка входу лабораторної роботи №8, варіант 9."""

from dataclasses import dataclass

from evaluator import count_nodes, depth, from_python, get, stats, with_field
from model import JSON, JNumber, JObject, JString
from pretty import pretty, render, tree


@dataclass(frozen=True)
class Car:
    """Довільний клас, що підходить під протокол SupportsToJSON."""

    model: str
    price: int

    def to_json(self) -> JSON:
        """JSON-представлення автомобіля."""
        return JObject((("model", JString(self.model)), ("price", JNumber(self.price))))


def run() -> None:
    """Демонструє ADT, match/case та операції над JSON-деревом."""
    doc = from_python(
        {
            "company": "Прокат авто",
            "active": True,
            "rating": 4.7,
            "manager": None,
            "cars": [Car("Skoda Octavia", 1200), Car("Toyota RAV4", 1900)],
            "tags": ["economy", "suv"],
        }
    )
    print("Компактно:", render(doc))
    print("\n3 відступами:\n" + pretty(doc))
    print("\nДерево:\n" + tree(doc))
    print(f"\nВузлів: {count_nodes(doc)}, глибина: {depth(doc)}")
    print(f"Статистика за типами: {stats(doc)}")
    print(f"cars[1].model = {render(get(doc, 'cars', 1, 'model') or JString('?'))}")
    if isinstance(doc, JObject):
        updated = with_field(doc, "rating", JNumber(4.9))
        print(
            f"Новий rating: {render(get(updated, 'rating') or JString('?'))}, ", end=""
        )
        print(f"старий не змінився: {render(get(doc, 'rating') or JString('?'))}")


def main() -> None:
    """Запуск демонстрації JSON-дерева."""
    run()


if __name__ == "__main__":
    main()
```

Файл tests/test_json_tree.py

test_json_tree.py рядки 1-59

```python
"""pytest-тести ЛР8, варіант 9: JSON-подібне дерево."""

import json
from dataclasses import FrozenInstanceError, dataclass

import pytest

from evaluator import (
    count_nodes,
    depth,
    from_python,
    get,
    kind,
    stats,
    to_python,
    with_field,
)
from model import JSON, JArray, JBool, JNull, JNumber, JObject, JString
from pretty import pretty, render, tree

SAMPLE = {
    "name": "Ivan",
    "age": 20,
    "score": 9.5,
    "student": True,
    "car": None,
    "langs": ["uk", "en", {"code": "pl", "level": [1, 2]}],
    "empty": {},
    "nothing": [],
}


@pytest.mark.parametrize(
    ("node", "expected_kind", "text"),
    [
        (JNull(), "null", "null"),
        (JBool(True), "bool", "true"),
        (JBool(False), "bool", "false"),
        (JNumber(42), "number", "42"),
        (JNumber(1.5), "number", "1.5"),
        (JString('Привіт "світ"'), "string", '"Привіт \\"світ\\""'),
        (JArray(), "array", "[]"),
        (JObject(), "object", "{}"),
    ],
)
def test_each_variant(node: JSON, expected_kind: str, text: str) -> None:
    """Тест: кожен варіант ADT має свій kind і рендер."""
    assert kind(node) == expected_kind
    assert render(node) == text
    assert count_nodes(node) == 1


def test_render_matches_json_module() -> None:
    """Тест: рендер вкладеної структури збігається з json.dumps."""
    doc = from_python(SAMPLE)
    assert render(doc) == json.dumps(SAMPLE, separators=(",", ":"), ensure_ascii=False)
    assert pretty(doc) == json.dumps(SAMPLE, indent=2, ensure_ascii=False)
```

test_json_tree.py рядки 60-121

```python
def test_round_trip() -> None:
    """Тест: from_python і to_python взаємно обернені."""
    assert to_python(from_python(SAMPLE)) == SAMPLE


def test_count_depth_stats_nested() -> None:
    """Тест: підрахунок вузлів, глибина і статистика для вкладеного дерева."""
    doc = from_python(SAMPLE)
    assert count_nodes(doc) == 16
    assert depth(doc) == 5
    assert stats(doc) == {
        "object": 3,
        "string": 4,
        "number": 4,
        "bool": 1,
        "null": 1,
        "array": 3,
    }


def test_get_by_path() -> None:
    """Тест: доступ за шляхом і відсутні шляхи."""
    doc = from_python(SAMPLE)
    assert get(doc, "langs", 2, "code") == JString("pl")
    assert get(doc, "langs", -1, "level", 0) == JNumber(1)
    assert get(doc, "langs", 10) is None
    assert get(doc, "missing") is None
    assert get(doc, "age", "x") is None
    assert get(doc) == doc


def test_with_field_is_pure() -> None:
    """Тест: with_field повертає новий об'єкт і не змінює старий."""
    obj = JObject((("a", JNumber(1)),))
    added = with_field(obj, "b", JNull())
    replaced = with_field(added, "a", JString("x"))
    assert obj == JObject((("a", JNumber(1)),))
    assert to_python(added) == {"a": 1, "b": None}
    assert to_python(replaced) == {"a": "x", "b": None}


def test_nodes_are_immutable() -> None:
    """Тест: вузли frozen і хешовані."""
    node = JString("x")
    with pytest.raises(FrozenInstanceError):
        node.value = "y"  # type: ignore[misc]
    assert len({JNumber(1), JNumber(1), JNull()}) == 2


def test_bool_is_not_number() -> None:
    """Тест: True перетворюється на JBool, а не JNumber."""
    assert from_python(True) == JBool(True)
    assert from_python(1) == JNumber(1)


def test_protocol_support() -> None:
    """Тест: будь-який об'єкт з to_json() підходить під SupportsToJSON."""

    @dataclass(frozen=True)
    class Point:
        """Точка з методом to_json."""
```

test_json_tree.py рядки 122-144

```
        x: int
        y: int

        def to_json(self) -> JSON:
            """JSON-представлення точки."""
            return JArray((JNumber(self.x), JNumber(self.y)))

    assert render(from_python({"p": Point(1, 2)})) == '{"p":[1,2]}'


def test_errors() -> None:
    """Тест: непідтримувані типи і нерядкові ключі."""
    with pytest.raises(TypeError):
        from_python({1, 2})
    with pytest.raises(TypeError):
        from_python({1: "a"})


def test_tree_view() -> None:
    """Тест: текстове дерево з типами вузлів."""
    assert tree(from_python({"a": [1, None]})) == (
        "root: object{1}\n  a: array[2]\n    [0]: 1\n    [1]: null"
    )
```

Запуск програми

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_08> py main.py
Компактно: {"company":"Прокат авто","active":true,"rating":4.7,"manager":null,"cars":[{"model":"Skod
a Octavia","price":1200},{"model":"Toyota RAV4","price":1900}],"tags":["economy","suv"]}

3 відступами:
{
  "company": "Прокат авто",
  "active": true,
  "rating": 4.7,
  "manager": null,
  "cars": [
    {
      "model": "Skoda Octavia",
      "price": 1200
    },
    {
      "model": "Toyota RAV4",
      "price": 1900
    }
  ],
  "tags": [
    "economy",
    "suv"
  ]
}

Дерево:
root: object{6}
  company: "Прокат авто"
  active: true
  rating: 4.7
  manager: null
  cars: array[2]
    [0]: object{2}
      model: "Skoda Octavia"
      price: 1200
    [1]: object{2}
      model: "Toyota RAV4"
      price: 1900
  tags: array[2]
    [0]: "economy"
    [1]: "suv"

Вузлів: 15, глибина: 4
Статистика за типами: {'object': 3, 'string': 5, 'bool': 1, 'number': 3, 'null': 1, 'array': 2}
cars[1].model = "Toyota RAV4"
Новий rating: 4.9, старий не змінився: 4.7
```

Рис. 1. Вивід програми (python main.py)

Виконання тестів

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_08> py -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe
rootdir: D:\IPZ2kurs\func\lab1\Lab_08
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.12.4
collecting ... collected 18 items

tests/test_json_tree.py::test_each_variant[node0-null-null] PASSED       [  5%]
tests/test_json_tree.py::test_each_variant[node1-bool-true] PASSED       [ 11%]
tests/test_json_tree.py::test_each_variant[node2-bool-false] PASSED      [ 16%]
tests/test_json_tree.py::test_each_variant[node3-number-42] PASSED       [ 22%]
tests/test_json_tree.py::test_each_variant[node4-number-1.5] PASSED      [ 27%]
tests/test_json_tree.py::test_each_variant[node5-string-"\u041f\u0440\u0438\u0432\u0456\u0442 \\"\u0441\u0432\u0456\u0442\\""] PASSED [ 33%]
tests/test_json_tree.py::test_each_variant[node6-array-[]] PASSED        [ 38%]
tests/test_json_tree.py::test_each_variant[node7-object-{}] PASSED       [ 44%]
tests/test_json_tree.py::test_render_matches_json_module PASSED          [ 50%]
tests/test_json_tree.py::test_round_trip PASSED                          [ 55%]
tests/test_json_tree.py::test_count_depth_stats_nested PASSED            [ 61%]
tests/test_json_tree.py::test_get_by_path PASSED                         [ 66%]
tests/test_json_tree.py::test_with_field_is_pure PASSED                  [ 72%]
tests/test_json_tree.py::test_nodes_are_immutable PASSED                 [ 77%]
tests/test_json_tree.py::test_bool_is_not_number PASSED                  [ 83%]
tests/test_json_tree.py::test_protocol_support PASSED                    [ 88%]
tests/test_json_tree.py::test_errors PASSED                              [ 94%]
tests/test_json_tree.py::test_tree_view PASSED                           [100%]

============================== 18 passed in 0.06s ==============================
```

Рис. 2. Результат запуску pytest

Перевірка типів і стилю

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_08> py -m mypy --strict .
Success: no issues found in 5 source files
PS D:\IPZ2kurs\func\lab1\Lab_08> py -m ruff check .
All checks passed!
PS D:\IPZ2kurs\func\lab1\Lab_08> py -m black --check .
All done!
5 files would be left unchanged.
```

Рис. 3. mypy, ruff та black без зауважень

Висновок:

Під час виконання роботи JSON-подібне дерево змодельовано як алгебраїчний тип даних із незмінних вузлів і рекурсивного TypeAlias. Уся логіка побудована на чистих функціях і match/case без ланцюгів isinstance, а Literal і

Protocol обмежують значення та описують структурні контракти. Код проходить mypy --strict, а збіг рендеру з модулем json підтверджено тестами.